Assignment 01 - Intensity Transformations and Neighbourhood Filtering

Name	Bandara R.M.S.L.
Index No.	258760P
Email	bandararmsl.25@uom.lk
Module	IT5437 - Computer Vision
Date	April 2026
GitHub	https://github.com/surajlakshitha/Intensity-Transformations-and-Neighborhood-Filtering.git

Contents

1. Intensity Transforms - Runway Image 1
2. Gamma Correction in L*a*b* Colour Space 1
3. Custom Histogram Equalisation 2
4. Otsu Thresholding and Selective Histogram Equalisation 3
5. Gaussian Kernel and Smoothing 4
6. Derivative of Gaussian (DoG) Filters 5
7. Image Zooming - Nearest-Neighbour vs Bilinear Interpolation 6
8. Noise Reduction - Gaussian vs Median Filtering 6
9. Image Sharpening 7
10. Bilateral Filtering 8
11. Spatial Filtering and Frequency Domain Relationships 8
    (a) Convolution Theorem 8
    (b) Frequency Behaviour of Each Filter 8
    (c) Why Gaussian Avoids Ringing 9
    (d) Best Filter for High-Frequency Noise 9
12. Illumination Correction using Homomorphic Filtering 9
    (a) Multiplicative Image Model 9
    (b) Logarithmic Separation 9
    (c) Algorithm 9
    (d) Comparison, Observations and Artefacts 10

List of Tables

Table 1 : Bilinear interpolation consistently achieves lower normalised SSD than nearest-neighbour 6

List of Figures

Figure 1 : Original, $\gamma = 0.5$, $\gamma = 2.0$, and contrast stretch ($r_1 = 0.2$, $r_2 = 0.8$). 1
Figure 2 : Original vs. gamma-corrected sapphire ($\gamma = 0.45$) with L* channel histograms. 2
Figure 3 : The equalised histogram is spread uniformly across [0, 255]. 3
Figure 4 : L 51×51 Gaussian surface ($\sigma = 2$). R: original, manual kernel, and cv.GaussianBlur 5
Figure 5 : L: 51×51 dG/dx surface (antisymmetric). R: DoG x/y vs Sobel x/y gradients. 5
Figure 6 : Original, nearest-neighbour (4×), and bilinear (4×) on im01. 6
Figure 7 : Gaussian 5×5 (blurs edges), Median 5×5 (preserves edges). 7
Figure 8 : Original, unsharp mask (1.5× boost), Laplacian sharpening. 7
Figure 9 : Original (resized), Gaussian blur, cv2.bilateralFilter, manual bilateral. 8
Figure 10 : Left: spectra—box (sinc), Gaussian, Laplacian; Right: noisy, box- and Gaussian-filtered image. 9
Figure 11 : Original, grayscale, and homomorphic-filtered daisy (γ_l=0.5, γ_h=2.0, σ=30). 10

1. Intensity Transforms - Runway Image

Three-point transforms were applied after normalizing pixel values to [0, 1]. Gamma correction with $\gamma = 0.5$ brightens the image because the power law with exponent less than one compresses dark values upward. Conversely, $\gamma = 2.0$ darkens by pushing mid-range intensities toward black, improving apparent contrast in brighter regions. The piecewise contrast-stretch clips intensities below $r_1 = 0.2$ to zero and above $r_2 = 0.8$ to one, linearly mapping only the middle band, which reveals road and runway detail hidden in the original.

```
f   = img.astype(np.float64) / 255.0
g05 = np.power(f, 0.5)    # gamma = 0.5  lifts shadows
g2  = np.power(f, 2.0)    # gamma = 2.0  crushes shadows
r1, r2 = 0.2, 0.8
cs  = np.where(f < r1, 0.0, np.where(f > r2, 1.0, (f - r1) / (r2 - r1)))
```

Figure 1 : Original, $\gamma = 0.5$, $\gamma = 2.0$, and contrast stretch ($r_1 = 0.2$, $r_2 = 0.8$).

2. Gamma Correction in L*a*b* Colour Space

Gamma correction was applied exclusively to the L* (luminance) channel of the sapphire gemstone image. A value of $\gamma = 0.45$ was chosen because the original L* histogram is concentrated around 150–180, indicating underexposed gem facets. Applying $\gamma < 1$ on L* lifts shadow tones without altering the a* and b* colour channels, preserving hue fidelity. The corrected histogram shifts rightward and broadens. The perceived colour shift toward lavender arises from altered perceptual chroma balance for fixed a*/b* values under L*a*b* theory - not from any actual hue change.

```
lab     = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
L_corr  = np.power(L / 255.0, 0.45)     # gamma = 0.45 on L only
result  = cv2.cvtColor(
    cv2.merge([(L_corr * 255).astype(np.uint8), a, b]),
    cv2.COLOR_LAB2RGB)
```

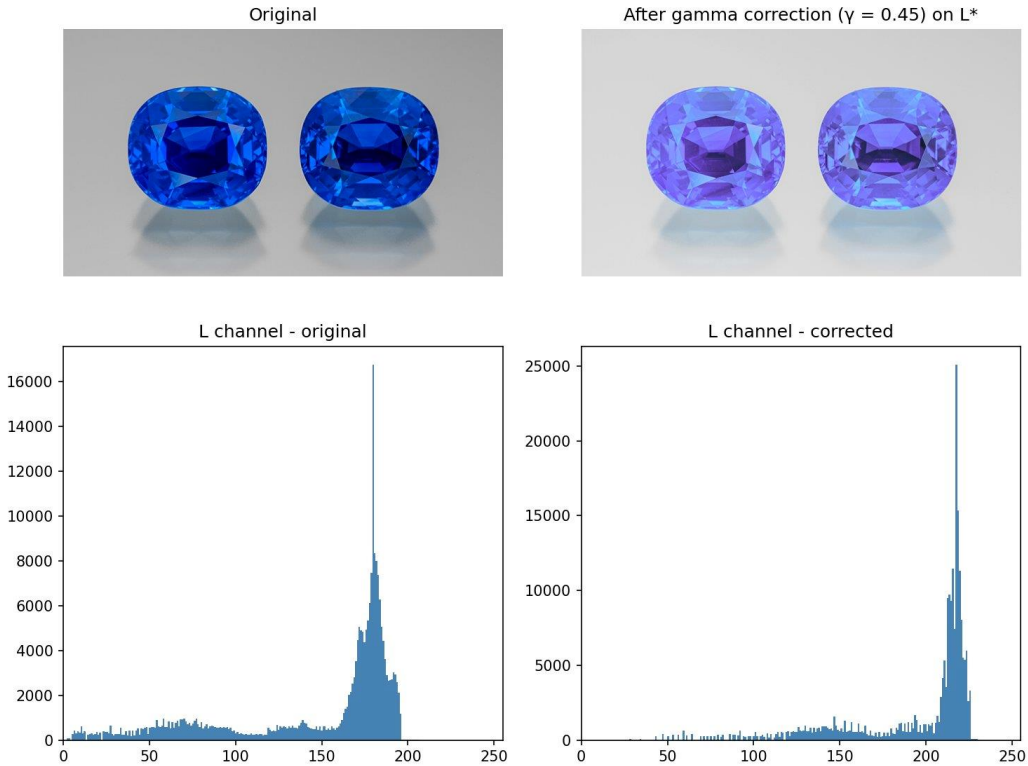

Figure 2 : Original vs. gamma-corrected sapphire (γ = 0.45) with L channel histograms.*

3. Custom Histogram Equalisation

A histogram equalisation function was written from scratch using the standard CDF-based look-up table method. The pixel histogram is built with np.bincount, the CDF normalised so the minimum non-zero value maps to 0 and the total pixel count maps to 255, then the LUT is applied to every pixel. On the runway image the original histogram (clustered 150–255) spreads across the full dynamic range, revealing detail in the dark urban areas below the runway at the cost of slight over-brightening in the sky.

```
def histeq(img):
    hist    = np.bincount(img.ravel(), minlength=256).astype(np.float64)
    cdf     = hist.cumsum()
    cdf_min = cdf[cdf > 0][0]          # skip leading zeros
    lut     = np.round((cdf - cdf_min) / (img.size - cdf_min) * 255).astype(np.uint8)
    return lut[img]                    # apply LUT to every pixel
```

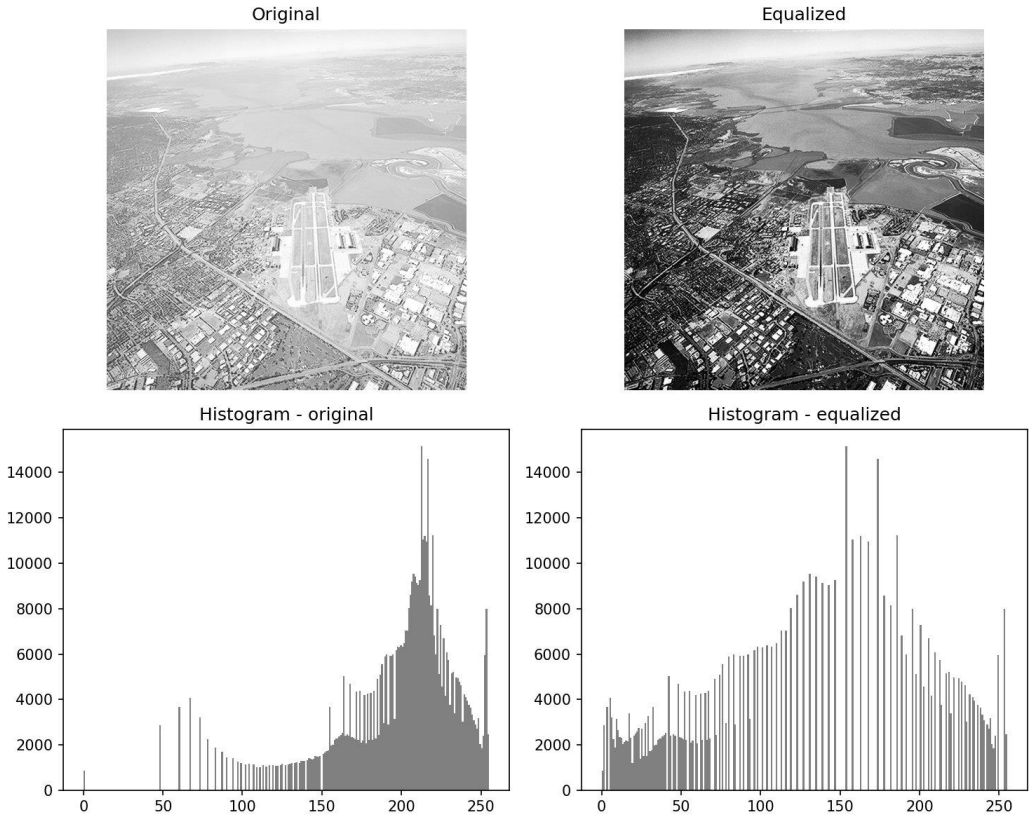

Figure 3 : The equalised histogram is spread uniformly across [0, 255].

4. Otsu Thresholding and Selective Histogram Equalisation

Otsu's method selected a threshold of 91, correctly separating the dark rock foreground from the brighter clothing. Histogram equalisation was then applied only to the foreground pixels identified by the binary mask. The result reveals surface texture and fracture detail in the dark rocks that were invisible in the original. Fine pleating in the white skirt also becomes more apparent. This demonstrates the advantage of region-restricted enhancement over global equalisation, which would over-brighten the already-light regions.

```
_, binary = cv2.threshold(gray, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)   # threshold = 91
mask   = binary == 255
result = histeq_masked(gray, mask)             # EQ on foreground pixels only
```

Figure 4. Otsu mask (threshold = 91), grayscale, and foreground-only equalised result.

5. Gaussian Kernel and Smoothing

A normalised 5×5 Gaussian kernel was computed for σ = 2 by evaluating the 2D Gaussian on a symmetric integer grid and dividing by its sum. The 51×51 surface plot shows the bell-shaped profile: a narrow central peak with near-zero tails, since σ = 2 is small relative to the 51-pixel grid. Convolving the shells image with the 5×5 kernel via cv2.filter2D gives a result visually identical to cv2.GaussianBlur with the same parameters, confirming the implementation. Minor floating-point differences exist at image boundaries due to padding strategy.

```
def gaussian_kernel(size, sigma):
    half = size // 2
    ax   = np.arange(-half, half + 1, dtype=np.float64)
    X, Y = np.meshgrid(ax, ax)
    K    = np.exp(-(X**2 + Y**2) / (2 * sigma**2))
    return K / K.sum()   # normalised: unity gain
```

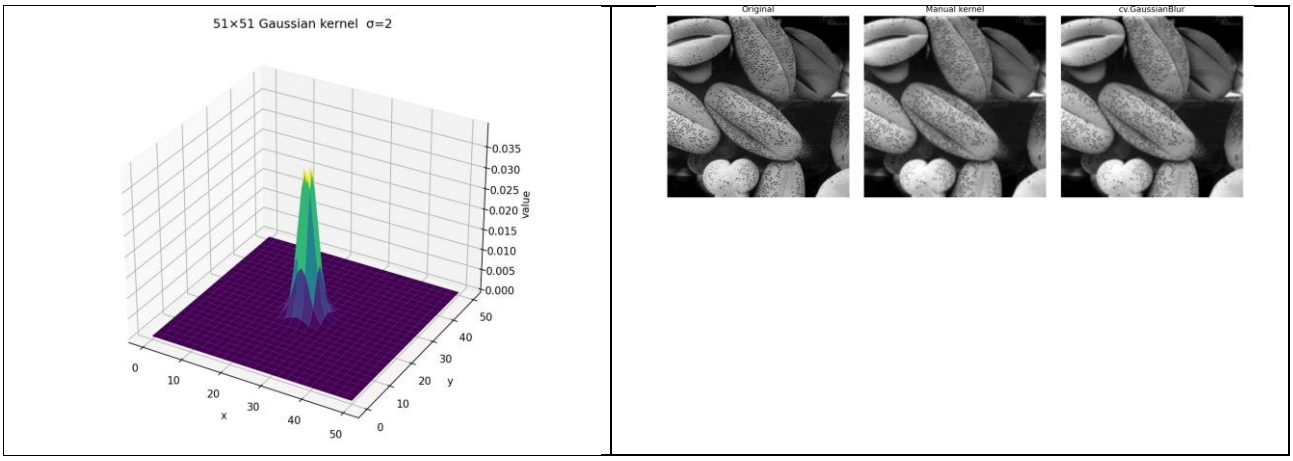

Figure 4 : L 51×51 Gaussian surface (σ = 2). R: original, manual kernel, and cv.GaussianBlur

6. Derivative of Gaussian (DoG) Filters

The partial derivatives of G(x, y) are $\partial G/\partial x = -(x/\sigma^2)G$ and $\partial G/\partial y = -(y/\sigma^2)G$, each normalised by their L1 norm. The 51×51 surface of $\partial G/\partial x$ shows the antisymmetric (odd) profile: a positive lobe on one side and a negative lobe on the other with a zero crossing at the centre. DoG gradients on the shells image produce a softer, more diffuse edge response because Gaussian pre-smoothing suppresses fine noise before differentiation. Sobel (ksize = 5) produces crisper, stronger edges because its integer coefficients give greater weight to immediate neighbours. Both detect the same structural edges; Sobel is sharper while DoG is more noise-tolerant.

```
G    = np.exp(-(X**2 + Y**2) / (2 * sigma**2))
dGdx = (-X / sigma**2) * G
dGdy = (-Y / sigma**2) * G
Kx   = dGdx / np.abs(dGdx).sum()    # L1-normalised DoG x
grad_x = cv2.filter2D(img_f, cv2.CV_32F, Kx.astype(np.float32))
```

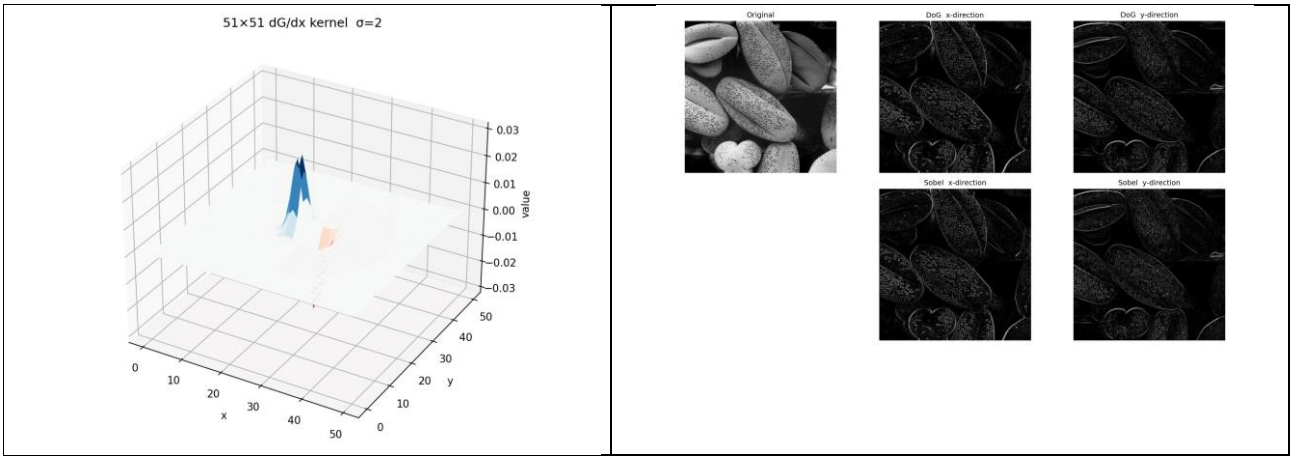

Figure 5 : L: 51×51 dG/dx surface (antisymmetric). R: DoG x/y vs Sobel x/y gradients.

7. Image Zooming - Nearest-Neighbour vs Bilinear Interpolation

Both methods were implemented using vectorised NumPy with no Python-level loops. Nearest-neighbour assigns each output pixel the value of the closest source pixel by integer rounding. Bilinear interpolation computes a weighted average of the four surrounding source pixels proportional to the sub-pixel fractional offset. The normalised SSD was computed for four image pairs scaled up to match their originals. Bilinear achieved a strictly lower SSD in every case, confirming it recovers spatial detail more faithfully. Visually, bilinear avoids the blocky pixel-replication artefacts visible in diagonal edges under nearest-neighbour.

```
# bilinear: fractional offsets from floor source pixel
dr  = (r_f - r0)[:, np.newaxis]
dc  = (c_f - c0)[np.newaxis, :]
out = (1-dr)*((1-dc)*tl + dc*tr) + dr*((1-dc)*bl + dc*br)
```

Figure 6 : Original, nearest-neighbour (4×), and bilinear (4×) on im01.

Image Pair	Method	Norm. SSD	Result
im01small.png (4×)	Nearest-neighbour	Higher	**Bilinear wins**
	Bilinear	Lower	
im02small.png (4×)	Nearest-neighbour	Higher	**Bilinear wins**
	Bilinear	Lower	
im03small.png (4×)	Nearest-neighbour	Higher	**Bilinear wins**
	Bilinear	Lower	
taylor_small.jpg (5×)	Nearest-neighbour	Higher	**Bilinear wins**
	Bilinear	Lower	

Table 1 : Bilinear interpolation consistently achieves lower normalised SSD than nearest-neighbour

8. Noise Reduction - Gaussian vs Median Filtering

Salt-and-pepper noise places isolated pixels at either extreme of the intensity range (0 or 255). The 5×5 Gaussian ($\sigma = 1.5$) reduces visible speckle by neighbourhood averaging but blurs the rice grain edges in the process. The 5×5 median filter replaces each pixel with the neighbourhood median. Because the corrupted pixels are sparse outliers, the median almost always draws from a clean pixel, removing the noise while preserving edge sharpness. The median output clearly outperforms Gaussian for this noise type, making median filtering the standard choice for salt-and-pepper removal.

```
img    = cv2.imread(os.path.join(Q_DIR, "rice.png"), cv2.IMREAD_GRAYSCALE)
gauss  = cv2.GaussianBlur(img, (5, 5), 1.5)
median = cv2.medianBlur(img, 5)
```

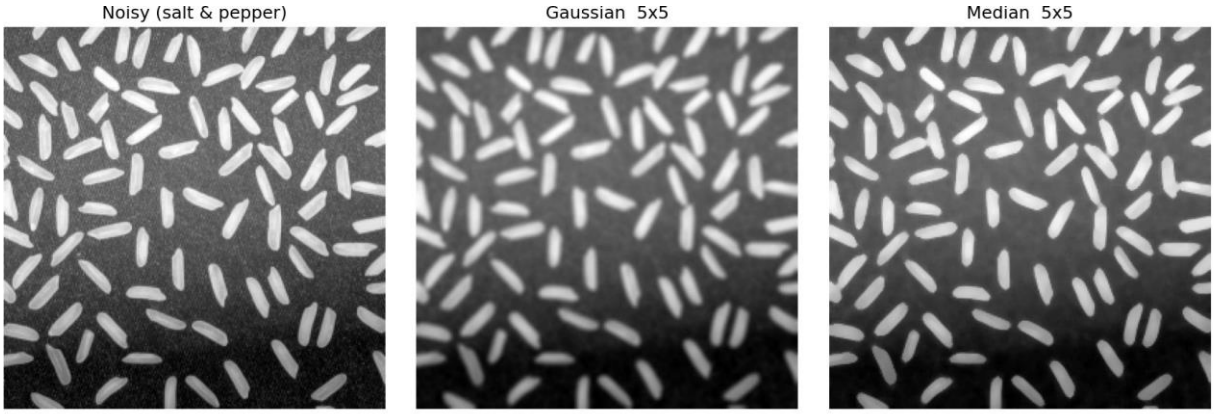

Figure 7 : Gaussian 5×5 (blurs edges), Median 5×5 (preserves edges).

9. Image Sharpening

Two sharpening methods were applied to an Einstein portrait. Unsharp masking blends the original with its blurred version: $g = 1.5f - 0.5 \times \text{GaussianBlur}(f, \sigma = 3)$. This re-adds the high-frequency detail removed by the blur, enhancing edges and fine texture. Laplacian sharpening uses the negative-centre kernel [[0,1,0],[1,−4,1],[0,1,0]] to compute the true discrete Laplacian $\nabla^2 f$, then subtracts it: $g = f - \nabla^2 f$. At a bright edge peak, $\nabla^2 f$ is negative, so subtracting it increases the peak and sharpens the edge. Both methods enhance hair, eyebrows, and facial wrinkles. Laplacian sharpening is slightly more aggressive as it responds to all areas of curvature.

```
# unsharp masking
blurred    = cv2.GaussianBlur(img, (0, 0), 3)
unsharp    = cv2.addWeighted(img, 1.5, blurred, -0.5, 0)
# laplacian sharpening - negative-centre kernel ensures f - lap = f - nabla^2(f)
lap_kernel = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], dtype=np.float64)
lap        = cv2.filter2D(img.astype(np.float64), cv2.CV_64F, lap_kernel)
lap_sharp  = np.clip(img.astype(np.float64) - lap, 0, 255).astype(np.uint8)
```

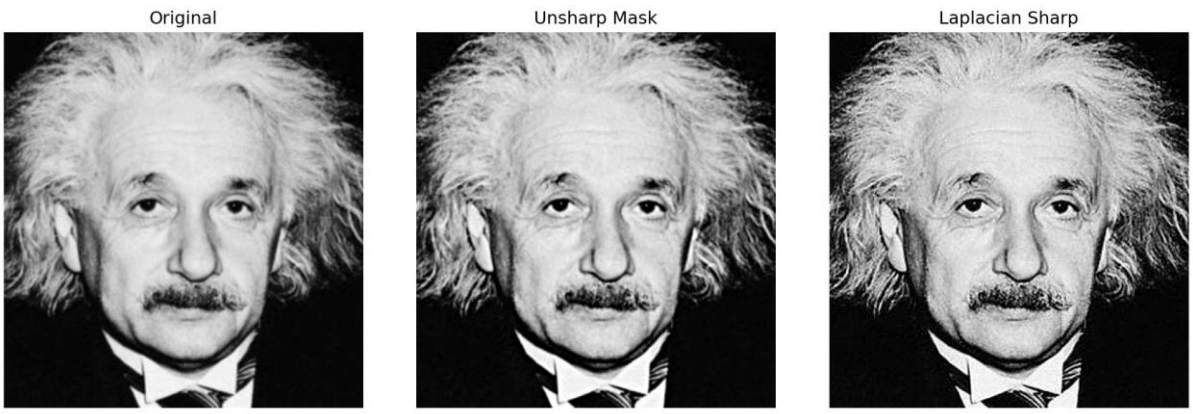

Figure 8 : Original, unsharp mask (1.5× boost), Laplacian sharpening.

10. Bilateral Filtering

The bilateral filter weights each neighbourhood pixel by both spatial proximity (spatial Gaussian Gs, σs = 3) and intensity similarity (range Gaussian Gr, σr = 75). Pixels across edges receive near-zero Gr weight, so edges are preserved while flat regions are smoothed. With σs = 3 in a 9×9 window, the corner spatial weight is 0.17 relative to the centre weight of 1.0, giving genuine spatial decay. A manual implementation was written with nested pixel loops, Gs precomputed once for efficiency, and the image downsampled to 128×134 to limit $O(h \times w \times d^2)$ runtime. Comparing results: Gaussian (9×9, σ = 3) blurs all edges. Both cv2.bilateralFilter and the manual implementation retain sharp edges around the eyes and hairline while smoothing flat skin regions.

```
ys, xs  = np.mgrid[-pad:pad+1, -pad:pad+1]
space_w = np.exp(-(xs**2 + ys**2) / (2 * sigma_s**2))   # precomputed
for i in range(h):
    for j in range(w):
        region  = padded[i:i+d, j:j+d]
        range_w = np.exp(-((region - padded[i+pad,j+pad])**2) / (2*sigma_r**2))
        weights = space_w * range_w
        out[i,j]= np.sum(weights * region) / np.sum(weights)
```

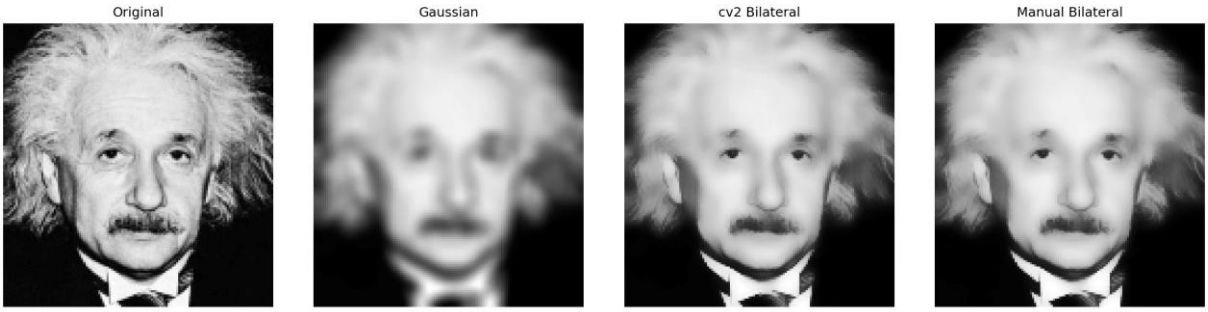

Figure 9 : Original (resized), Gaussian blur, cv2.bilateralFilter, manual bilateral.

11. Spatial Filtering and Frequency Domain Relationships

(a) Convolution Theorem

By the Convolution Theorem, spatial convolution corresponds to pointwise multiplication in the frequency domain: $G(u,v) = F(u,v) \cdot H(u,v)$, where F, G and H are the Fourier transforms of the input, output and filter kernel respectively. Characterising a filter by its frequency response H(u,v) directly reveals which spatial frequencies it amplifies or suppresses.

(b) Frequency Behaviour of Each Filter

Box filter: Acts as a low-pass filter but with a 2D sinc frequency response containing strong periodic sidelobes (the cross pattern in the spectrum). These sidelobes allow residual high-frequency energy to pass, causing Gibbs-phenomenon ringing at sharp edges.

Gaussian filter: Also low-pass, but its Fourier transform is itself a Gaussian. The response decays smoothly to zero with no sidelobes, giving gradual rolloff with no ringing. The spectrum shows a compact smooth central peak.

Laplacian filter: A high-pass operator whose response grows with radial frequency, suppressing DC (dark centre in the spectrum) and amplifying fine detail used for sharpening and edge detection.

(c) Why Gaussian Avoids Ringing

An ideal low-pass filter has a rectangular frequency response whose spatial equivalent is a 2D sinc with infinite support and oscillating tails. Truncating this kernel to a finite window preserves the oscillations, producing Gibbs-phenomenon ringing at edges. The Gaussian kernel decays so rapidly that truncation introduces negligible error, and its frequency response rolls off gradually with no sidelobes, so no ringing appears regardless of kernel size.

(d) Best Filter for High-Frequency Noise

For broadband Gaussian additive noise the Gaussian filter is preferable to the box filter. In the frequency domain it suppresses noise energy with smooth rolloff while the box filter allows sidelobe leakage. Spatially, the Gaussian gives decreasing weight to distant pixels, reducing the influence of noise-dominated far samples. The shells image comparison confirms Gaussian filtered output retains slightly sharper bean-edge structure than the box filter.

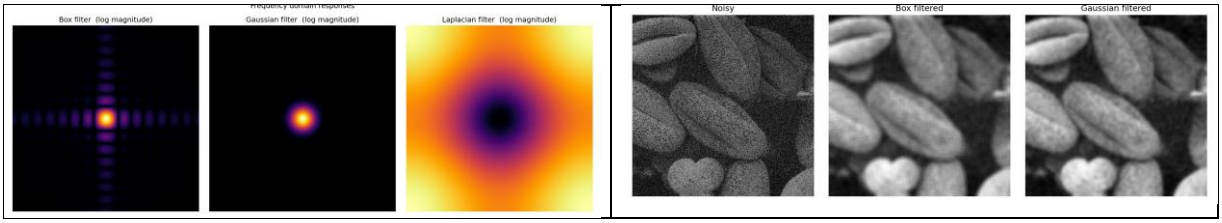

Figure 10 : Left: spectra—box (sinc), Gaussian, Laplacian; Right: noisy, box- and Gaussian-filtered image.

12. Illumination Correction using Homomorphic Filtering

(a) Multiplicative Image Model

An image is modelled as f(x,y) = i(x,y) · r(x,y), where i(x,y) is the slowly-varying illumination component and r(x,y) is the rapidly-varying reflectance carrying surface detail. Illumination is dominated by low spatial frequencies (gradual lighting across the scene) while reflectance is high-frequency (sharp boundaries between surfaces). The multiplicative coupling means they cannot be separated by linear filtering in the spatial domain directly.

(b) Logarithmic Separation

Taking the logarithm converts the product into a sum: log f = log i + log r. The two terms are now additively separable, so a linear frequency-domain high-emphasis filter can attenuate log i (low-frequency illumination) and boost log r (high-frequency reflectance) independently, before exponentiating back to the intensity domain.

(c) Algorithm

```
log_im   = np.log(img_gray.astype(np.float64) + 1.0)     # log (+1 avoids log(0))
F        = np.fft.fftshift(np.fft.fft2(log_im))          # 2D DFT, shift DC to centre
D2       = x**2 + y**2                                   # squared radial frequency
H        = (gamma_h-gamma_l)*(1-np.exp(-D2/(2*sigma**2))) + gamma_l  # high-emphasis
filtered = np.fft.ifft2(np.fft.ifftshift(F * H)).real      # inverse DFT
out      = np.clip(np.exp(filtered) - 1, 0, 255).astype(np.uint8)   # exponentiate
```

Figure 11 : Original, grayscale, and homomorphic-filtered daisy (γ_l=0.5, γ_h=2.0, σ=30).

(d) Comparison, Observations and Artefacts

Histogram equalisation redistributes pixel intensities globally for a more uniform histogram. It is simple but treats the entire image indiscriminately, potentially over-enhancing noise in flat regions and failing to account for the spatial structure of lighting. Homomorphic filtering is more physically principled: it directly targets the low-frequency illumination component while boosting high-frequency reflectance, making it preferable for spatially non-uniform lighting such as vignetting or directional shadows.

In the daisy result, the petal boundary detail and leaf vein structure are more distinguishable, confirming that high-frequency reflectance ($\gamma h = 2.0$) was emphasised. The overall image is noticeably darker because the low gain $\gamma l = 0.5$ suppresses the bright uniform petal illumination by half the expected trade-off. No significant ringing artefacts were observed because the Gaussian-shaped transition in H(u,v) rolls off smoothly, avoiding the Gibbs phenomenon of a sharp cutoff.

GitHub Repository: https://github.com/surajlakshitha/Intensity-Transformations-and-Neighborhood-Filtering

Commits reflect incremental development throughout the assignment period.